

算法更高。

4. 反向删除算法

反向删除算法是一种基于贪心策略的最小生成树算法，其思想是首先按照边的权值从大到小对所有边进行排序，然后逐步删除边，直到图不再连通为止。在每次删除边的过程中，都需要判断该边的删除是否会导致图不再连通，如果会则保留该边，否则删除该边。

具体步骤如下：

(1) 将所有边按照边权从大到小排序。

(2) 依次遍历所有边，对于每条边，先将其从图中删除，然后检查图是否仍然连通，如果不连通，则将该边加回图中，否则保留该边的删除操作。

(3) 重复执行步骤(2)，直到图不再连通为止。

反向删除算法的时间复杂度为 $O(E \log V)$ ，其中 E 是边的数量。该算法相对于其他最小生成树算法的优点在于其实现简单，且可以在大多数情况下得到接近于最优的结果，但是在稠密图上可能会出现较差的效果。

2.2.2 最短路径

带权图的最短路径问题即求两个顶点间长度最短的路径。其中路径长度不是指路径上边数的总和，而是指路径上各边的权值总和。路径长度的具体含义取决于边上权值所代表的意义。下面介绍几种求带权图的最短路径的算法。

1. 迪杰斯特拉算法

已知有向带权图 $G=(V, E)$ ，找出从某个源点 $s \in V$ 到 V 中其余各顶点的最短路径，称为单源最短路径。

目前，求单源最短路径主要使用迪杰斯特拉 (Dijkstra) 提出的一种按路径长度递增次序产生各顶点最短路径的算法。若按长度递增的次序生成从源点 s 到其他顶点的最短路径，则当前正在生成的最短路径上除终点以外，其余顶点的最短路径均已生成（将源点的最短路径看作已生成的源点到其自身的长度为 0 的路径）。

迪杰斯特拉算法的基本思想是：设 S 为最短距离已确定的顶点集（看作红点集）， $V-S$ 是最短距离尚未确定的顶点集（看作蓝点集）。具体步骤如下：

(1) 初始化：初始化时，只有源点 s 的最短距离是已知的 ($SD(s)=0$)，故红点集 $S=\{s\}$ ，蓝点集包含除源点 s 之外的其余所有顶点。

(2) 重复以下工作：在当前蓝点集中选择一个最短距离最小的蓝点来扩充红点集，以保证算法按路径长度递增的次序产生各顶点的最短路径。当蓝点集中只剩下最短距离为 ∞ 的蓝点，或者所有蓝点已扩充到红点集时， s 到所有顶点的最短路径就求出来了。

需要注意的是：

(1) 若从源点到蓝点的路径不存在，则可假设该蓝点的最短路径是一条长度为无穷大的虚拟路径。

(2) 从源点 s 到终点 t 的最短路径简称为 t 的最短路径； s 到 t 的最短路径长度简称为 t 的最

短距离, 记为 $SD(i)$ 。

根据按长度递增次序产生最短路径的思想, 当前最短距离最小的蓝点 k 的最短路径是:

源点, 红点 1, 红点 2, ..., 红点 n , 蓝点 k

距离为: 源点到红点 n 的最短距离 + $\langle$ 红点 n , 蓝点 $k \rangle$ 的边长

为求解方便, 可设置一个向量 $D[0..n-1]$, 对于每个蓝点 $v \in V-S$, 用 $D[v]$ 记录从源点 s 到达 v 且除 v 外中间不经过任何蓝点 (若有中间点, 则必为红点) 的“最短”路径长度 (简称估计距离)。若 k 是蓝点集中估计距离最小的顶点, 则 k 的估计距离就是最短距离, 即若 $D[k] = \min\{D[i] \mid i \in V-S\}$, 则 $D[k] = SD(k)$ 。

初始时, 每个蓝点 v 的 $D[v]$ 值应为权 $w\langle s, v \rangle$, 且从 s 到 v 的路径上没有中间点, 因为该路径仅含一条边 $\langle s, v \rangle$ 。

将 k 扩充到红点集后, 剩余蓝点集中顶点的估计距离可能由于增加了新红点 k 而减小, 此时必须调整相应蓝点的估计距离。对于任意的蓝点 j , 若 k 由蓝变红后使 $D[j]$ 变小, 则必定是由于存在一条从 s 到 j 且包含新红点 k 的更短路径: $P = \langle s, \dots, k, j \rangle$ 。且 $D[j]$ 减小的新路径 P 只可能是由路径 $\langle s, \dots, k \rangle$ 和边 $\langle k, j \rangle$ 组成。所以, 当 $\text{length}(P) = D[k] + w\langle k, j \rangle$ 小于 $D[j]$ 时, 应该用 P 的长度来修改 $D[j]$ 的值。

2. 弗洛伊德算法

对图中的每对顶点 u 和 v , 找出 u 到 v 的最短路径问题。在实际应用中, 这一问题可用每个顶点作为源点调用一次单源最短路径问题的迪杰斯特拉算法予以解决, 但更常用的是弗洛伊德 (Floyd) 提出的求每一对顶点之间的最短路径的算法。

弗洛伊德算法是一种动态规划算法, 用于解决任意两点间的最短路径问题。算法的基本思路是, 通过枚举中间节点来逐步更新节点间的最短路径。算法的流程如下:

(1) 初始化距离矩阵 d , $d[i][j]$ 表示节点 i 到节点 j 的最短路径长度, 如果节点 i 和节点 j 之间没有边相连, 则 $d[i][j]$ 为正无穷。

(2) 对于每个节点对 (i, j) , 如果存在一条从节点 i 到节点 j 的边, 则将 $d[i][j]$ 赋值为该边的权值, 否则将 $d[i][j]$ 赋值为正无穷。

(3) 对于每个节点对 (k, i, j) , 如果从节点 i 到节点 j 经过节点 k 的路径比当前从 i 到 j 的路径更短, 则更新 $d[i][j]$ 的值。

(4) 重复执行步骤 (3), 直到所有的节点对的最短路径被计算出来。

3. 贝尔曼 - 福特算法

贝尔曼 - 福特 (Bellman-Ford) 算法是一种用于解决带有负权边的最短路径问题的动态规划算法。它可以求解从单个源节点到图中所有节点的最短路径, 并且能够检测负权回路。该算法的基本思路是利用松弛操作来逐步更新每个节点的最短路径估计值, 同时使用动态规划的思想, 递推求解最短路径问题。

具体实现时, 贝尔曼 - 福特算法需要进行 $n-1$ 轮松弛操作, 其中 n 是图中节点的个数。在每一轮中, 对于每个边 (u, v) , 都要尝试使用 u 的最短路径更新 v 的最短路径估计值。如果某个节点的最短路径估计值在第 n 轮仍然发生了变化, 则说明存在负权回路。

贝尔曼-福特算法的时间复杂度为 $O(nm)$ ，其中 n 是节点数， m 是边数。与迪杰斯特拉算法相比，贝尔曼-福特算法更加通用，但由于其时间复杂度较高，不适用于边数较多的情况。

4. A* 算法

A* (A-Star) 算法是一种启发式搜索算法，常用于解决图上的最短路径问题。它通过估计从当前节点到目标节点的最短距离来指导搜索方向，从而加快搜索速度。A* 算法基于启发式函数，该函数将节点从当前位置到目标位置的估计最短距离与从起点到该节点的已知距离相结合，计算出节点的估计距离。然后，在搜索时优先考虑估计距离较短的节点，以尽快找到最短路径。

具体实现时，A* 算法使用一个优先队列来存储候选节点，每次从队列中取出估计距离最小的节点进行扩展，直到找到目标节点为止。在扩展节点时，需要考虑与该节点相邻的所有节点，并计算它们的估计距离，然后将它们加入到优先队列中。

A* 算法的时间复杂度与启发式函数有关，通常情况下为 $O(b^d)$ ，其中 b 是每个节点的平均分支因子， d 是起点到目标节点的最短距离。与迪杰斯特拉算法和贝尔曼-福特算法相比，A* 算法能够更快地找到最短路径，但由于需要计算启发式函数，所以在某些情况下可能会失效。

5. 约翰逊算法

约翰逊 (Johnson) 算法是一种用于解决有向图上最短路径问题的算法，它结合了迪杰斯特拉算法和贝尔曼-福特算法的优点，能够在时间复杂度为 $O(VE \log V)$ 的情况下求出所有节点之间的最短路径。

约翰逊算法的基本思路如下：

(1) 首先，对原始有向图进行转换，添加一个新的节点 s ，使得 s 到所有节点的边权值均为 0，并且将所有边权值都变成非负数。

(2) 使用贝尔曼-福特算法计算出 s 到所有节点的最短路径，如果有负权环，则算法结束，否则继续进行。

(3) 对于原始有向图中的每一条边 (u, v) ，将其边权值更新为原始边权值加上 s 到 u 的最短路径减去 s 到 v 的最短路径。

(4) 对每个节点 u 运行迪杰斯特拉算法，计算出它到所有其他节点的最短路径，这个过程中使用了更新后的边权值。

(5) 最后，将所有节点之间的最短路径合并起来，得到原始有向图中所有节点之间的最短路径。

总的来说，约翰逊算法的思路是先利用贝尔曼-福特算法找到图中所有节点到源点 s 的最短路径，然后根据这个最短路径的结果，调整边的权重，使得原图中的所有负边变成非负边。接着，使用迪杰斯特拉算法计算每个节点到其他所有节点的最短路径，最后再将迪杰斯特拉算法的结果合并起来，得到原始有向图中所有节点之间的最短路径。

6. SPFA 算法

SPFA (Shortest Path Faster Algorithm) 算法是一种用于解决带有负权边的图上最短路径问题的算法。它是一种基于贝尔曼-福特算法的改进算法，通过避免重复的松弛操作，使得算法的时间复杂度比贝尔曼-福特算法更优。

SPFA 算法的基本思路如下：

(1) 初始化距离数组 $dis[]$ 为无穷大，起点 s 的距离 $dis[s]$ 为 0。

(2) 将起点 s 加入队列 Q 中。

(3) 对于队列中的每一个节点 u ，依次遍历其所有邻居节点 v ，并对 $dis[v]$ 进行松弛操作（即尝试通过 u 到达 v ，判断是否可以更新 v 的最短距离），如果发现可以更新的最短距离，则将 v 加入队列 Q 中。

(4) 对队列 Q 中的节点进行循环操作，直到队列为空。在循环的过程中，需要避免重复地对同一节点进行松弛操作。因此，可以维护一个数组 $inqueue[]$ ，表示一个节点是否已经在队列中。

(5) 循环结束后， $dis[]$ 数组中保存了起点 s 到所有节点的最短距离。

SPFA 算法的时间复杂度最坏情况下为 $O(VE)$ ，但是在实际应用中，由于算法会剪枝，避免对同一节点进行重复的松弛操作，因此实际运行效率往往比较高。

2.2.3 网络与最大流量

许多应用中包含流量问题。例如，公路系统中有车辆流，控制系统中有信息流，网络系统中有数据流，金融系统中有现金流等。在实际应用中，很多时候需要求解最大流量问题。

最大流量问题是一个特殊的线性规划问题。为了便于读者理解和解答相关问题，本节不介绍有关网络与最大流量的理论知识，而是通过一个实际例子来说明最大流量问题的基本概念和解答方法。

图 2-1 标出了某地区的运输网，各节点之间的运输能力如表 2-1 所示。

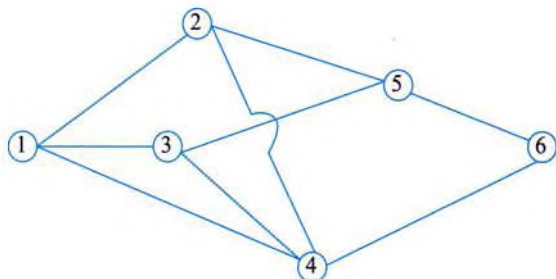


图 2-1 某地区的运输网

表 2-1 各节点之间的运输能力

（单位：万吨 / 小时）

节点	①	②	③	④	⑤	⑥
①		6	10	10		
②	6			4	7	
③	10			1	14	
④	10	4	1			5
⑤		7	14			21
⑥				5	21	

要计算从节点①到节点⑥的最大运输能力（流量），首先把表 2-1 中的数据标记到图 2-1 上，形成图 2-2。

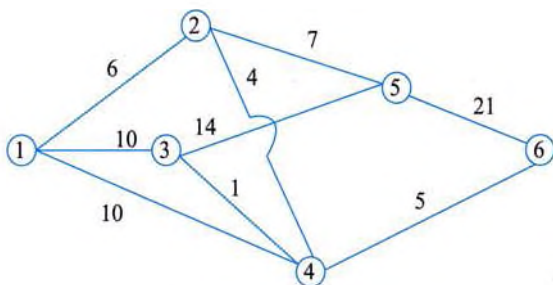


图 2-2 新的运输网

在运输网络的实际问题中可以看出，对于流有两个明确的要求，一是每条边（弧）上的流量不能超过该边的最大通过能力（即边的容量），二是中间节点的流量为 0。对于每个节点，运出这个节点的产品总量与运进这个节点的产品总量之差，是这个节点的净输出量，简称为这个节点的流量。由于中间节点只起到转运作用，所以中间节点的流量为 0。另外，起始点的净流出量和终点的净流入量必须相等，也就是这个方案的总运输量。

在本例中，从节点①到节点⑥可以同时沿多条路径运输，总的最大流量应是各条路径上的最大流量之和，每条路径上的最大流量应是其各段流量的最小值。

计算时，每找出一条路径算出流量后，该路径上各段线路上的流量应扣除已经算过的流量，形成剩余流量。剩余流量为 0 的线段应将其删除（断开）。这种做法比较简单。例如，路径①③⑤⑥的最大流量为 10 万吨，计算过后，该路径上各段流量应都减少 10 万吨。从而①③之间将断开，③⑤之间的剩余流量是 4 万吨，⑤⑥之间的剩余流量为 11 万吨，如图 2-3 所示。

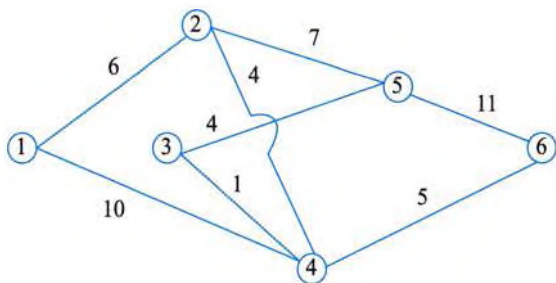


图 2-3 ①③断开后的运输网

同理，依次执行类似的步骤：

(1) 路径①②⑤⑥的剩余最大流量为 6 万吨。计算过后，该路径上各段流量应都减少 6 万吨。从而①②之间将断开，②⑤之间的剩余流量是 1 万吨，⑤⑥之间的剩余流量为 5 万吨，如图 2-4 所示。

(2) 路径①④⑥的剩余最大流量为 5 万吨。计算过后，该路径上各段流量应都减少 5 万吨。从而④⑥之间将断开，①④之间的剩余流量是 5 万吨，如图 2-5 所示。

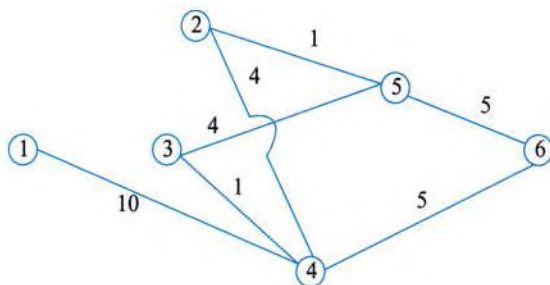


图 2-4 ①②断开后的运输网

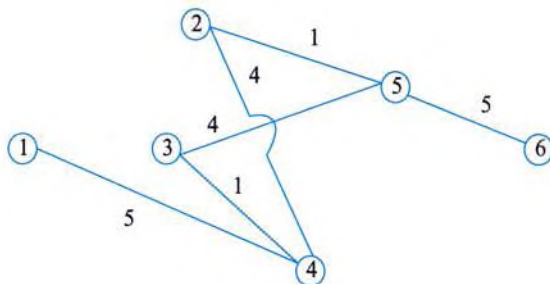


图 2-5 ④⑥断开后的运输网

(3) 路径①④③⑤⑥的剩余最大流量为 1 万吨。计算过后，该路径上各段流量应都减少 1 万吨。从而④③之间将断开，①④之间的剩余流量是 4 万吨，③⑤之间的剩余流量为 3 万吨，⑤⑥之间的剩余流量为 4 万吨，如图 2-6 所示。

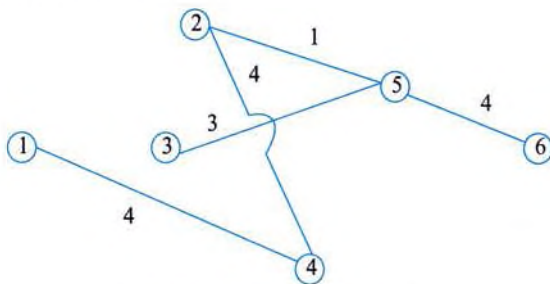


图 2-6 ④③断开后的运输网

(4) 路径①④②⑤⑥的剩余最大流量为 1 万吨。计算过后，该路径上各段流量应都减少 1 万吨。从而②⑤之间将断开，①④之间、④②之间、⑤⑥之间的剩余流量都是 3 万吨，如图 2-7 所示。

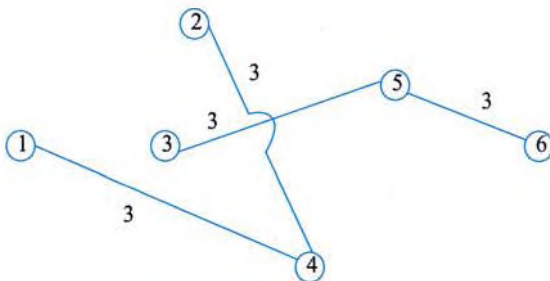


图 2-7 ②⑤断开后的运输网